

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

Hands-On #12

Mini-shell in C: `fork`, `exec`, `wait` e le pipe

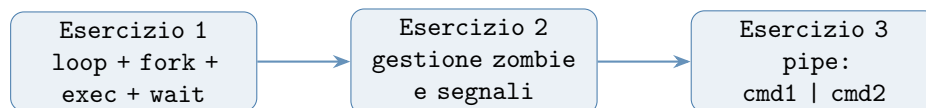
Last Update: 29 aprile 2026

Scenario

In questo Hands-On costruiremo una mini-shell in C, partendo dalle primitive più semplici e aggiungendo funzionalità passo dopo passo. Al termine avremo un programma che legge comandi da tastiera, li esegue come processi figli, gestisce correttamente la terminazione, e supporta l'operatore **pipe** (`|`) per connettere due comandi.

Non useremo `system()` né `popen()`: tutto verrà costruito a mano con `fork()`, `execvp()`, `waitpid()`, `pipe()` e `dup2()`.

Il percorso è suddiviso in tre esercizi progressivi:



Concetti necessari

Prima di scrivere il codice, ripassiamo le primitive che useremo.

`fork()` e il processo figlio

`fork()` crea una copia esatta del processo chiamante. Ritorna due volte: **0** nel figlio, il **PID del figlio** nel padre, **-1** in caso di errore. Padre e figlio condividono le stesse pagine fisiche finché uno dei due non scrive (*copy-on-write*): la copia è quasi gratuita.

`execvp()` e la famiglia `exec`

`execvp(file, argv)` sostituisce l'immagine del processo corrente con un nuovo programma, cercandolo nel **PATH**. Se ha successo **non ritorna mai**: il codice che segue la **exec** viene eseguito solo in caso di errore. Il vettore `argv` deve essere terminato da **NULL**.

`waitpid()` e gli zombie

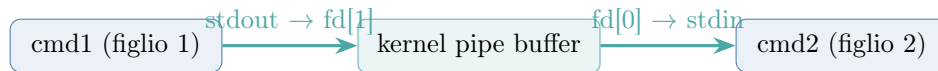
Quando un figlio termina, la sua `task_struct` rimane in memoria (stato **Z** — **zombie**) finché il padre non chiama `wait()` o `waitpid()`. Ogni shell ben scritta deve raccogliere i figli per non lasciare zombie in giro.

`waitpid(pid, &status, 0)` attende un figlio specifico. `waitpid(-1, &status, WNOHANG)` è non bloccante: ritorna subito se nessun figlio è terminato.

`pipe()`, `dup2()` e il reindirizzamento

`pipe(fd)` crea un canale unidirezionale: `fd[0]` è il capo in lettura, `fd[1]` il capo in scrittura. I byte scritti in `fd[1]` escono da `fd[0]`.

`dup2(oldfd, newfd)` duplica `oldfd` su `newfd`, chiudendo prima `newfd` se era già aperto. Serve per collegare il capo della pipe a `STDIN_FILENO` o `STDOUT_FILENO` prima di chiamare `exec`.



[!] Chiudere sempre i capi inutilizzati

Dopo la `fork()`, sia il padre che il figlio hanno copie di `fd[0]` e `fd[1]`. Il processo che scrive deve chiudere `fd[0]`; quello che legge deve chiudere `fd[1]`. Se non lo fa, il lettore non riceverà mai EOF: la pipe rimane aperta finché esiste almeno un descrittore di scrittura.

Esercizio 1 — La shell base

Esercizio 1: loop read-fork-exec-wait

Scrivi un programma `minish.c` che:

1. Stampa un prompt `minish>` e legge una riga da `stdin` con `fgets()`.
2. Fa il parsing della riga in un vettore `argv[]` (usa `strtok()`).
3. Chiama `fork()`: nel figlio esegue il comando con `execvp()`; nel padre aspetta con `waitpid()`.
4. Se l'utente digita `exit`, il programma termina.
5. Gestisce correttamente il caso in cui `execvp()` fallisce (comando non trovato): il figlio deve uscire con `exit(127)`.

Di seguito lo scheletro con i punti chiave già commentati. Completa le parti mancanti indicate da `/* TODO */`.

Listing 1: `minish.c` — scheletro Esercizio 1

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/types.h>
6  #include <sys/wait.h>
7
8  #define MAXLINE 1024
9  #define MAXARGS 64
10
11 /* Tokenizza 'line' in argv[], ritorna il numero di argomenti. */
12 static int parse(char *line, char *argv[]) {
13     int argc = 0;
14     char *tok = strtok(line, " \t\n");
15     while (tok && argc < MAXARGS - 1) {
16         argv[argc++] = tok;
17         tok = strtok(NULL, " \t\n");
18     }
19     argv[argc] = NULL; /* termine obbligatorio per execvp */
20     return argc;
21 }
22
23 int main(void) {
24     char line[MAXLINE];
25     char *argv[MAXARGS];
26

```

```

27     while (1) {
28         /* 1. Stampa il prompt e leggi la riga */
29         printf("minish> ");
30         fflush(stdout);
31         if (!fgets(line, MAXLINE, stdin))
32             break; /* EOF: Ctrl-D */
33
34         /* 2. Parsing */
35         if (parse(line, argv) == 0)
36             continue; /* riga vuota */
37
38         /* 3. Comando built-in: exit */
39         if (strcmp(argv[0], "exit") == 0)
40             break;
41
42         /* 4. Fork */
43         pid_t pid = fork();
44         if (pid < 0) {
45             perror("fork");
46             continue;
47         }
48
49         if (pid == 0) {
50             /* --- FIGLIO --- */
51             /* TODO: chiama execvp(argv[0], argv).
52              * Se fallisce, stampa un messaggio di errore
53              * con perror() ed esci con exit(127). */
54
55         } else {
56             /* --- PADRE --- */
57             /* TODO: attendi la terminazione del figlio con
58              * waitpid(). Stampa il codice di uscita se != 0. */
59         }
60     }
61     return 0;
62 }

```

► Come estrarre il codice di uscita

`waitpid()` riempie un intero `status`. Usa le macro:

- `WIFEXITED(status)` — vero se il figlio è terminato normalmente
- `WEXITSTATUS(status)` — codice passato a `exit()`
- `WIFSIGNALED(status)` — vero se il figlio è stato ucciso da un segnale
- `WTERMSIG(status)` — numero del segnale

Test attesi:

```

$ gcc -Wall -Wextra -o minish minish.c
$ ./minish
minish> ls -l
total 16
-rwxr-xr-x 1 user user 12345 Apr 29 10:00 minish
-rw-r--r-- 1 user user 2048 Apr 29 09:55 minish.c
minish> echo ciao mondo
ciao mondo
minish> comando_inesistente
execvp: No such file or directory
[exit 127]
minish> exit

```

Esercizio 2: gestione zombie e built-in cd

Estendi `minish.c` con:

1. **Comandi in background:** se la riga termina con `&`, il padre *non* aspetta il figlio (esecuzione in background). Installa un handler per `SIGCHLD` che chiami `waitpid(-1, NULL, WNOHANG)` per raccogliere automaticamente i figli terminati ed evitare zombie.
2. **Built-in cd:** il cambio di directory deve avvenire nel processo della shell stessa (non in un figlio), quindi va gestito con `chdir()` *prima* di fare `fork`. Implementa `cd [dir]` e `cd` senza argomenti (va in `HOME`).
3. **Stampa PID del figlio** per i comandi in background, come fa bash: `[1] 4321`.

▷ Perché cd non può essere un processo figlio?

`chdir()` modifica la directory corrente del processo che la chiama. Se la shell fa `fork()` e poi il figlio fa `chdir()`, cambia la directory del figlio — non della shell. Quando il figlio termina, la shell è ancora nella directory di partenza. Tutti i comandi che modificano lo stato della shell (`cd`, `export`, `exit`, `ulimit`...) devono essere *built-in*, cioè eseguiti direttamente dalla shell senza fare `fork`.

▷ SIGCHLD e raccolta asincrona

Quando un figlio termina, il kernel manda `SIGCHLD` al padre. Se il padre ha un handler che chiama `waitpid(-1, NULL, WNOHANG)` in un loop, raccoglie *tutti* i figli terminati senza bloccarsi. Il loop è necessario perché più figli possono terminare mentre il segnale è mascherato.

```
1 static void sigchld_handler(int sig) {
2     (void)sig;
3     while (waitpid(-1, NULL, WNOHANG) > 0)
4         ; /* raccoglie tutti i figli pronti */
5 }
```

Test attesi:

```
minish> sleep 5 &
[1] 5678
minish> pwd
/home/user
minish> cd /tmp
minish> pwd
/tmp
minish> cd
minish> pwd
/home/user
```

Esercizio 3 — La pipe

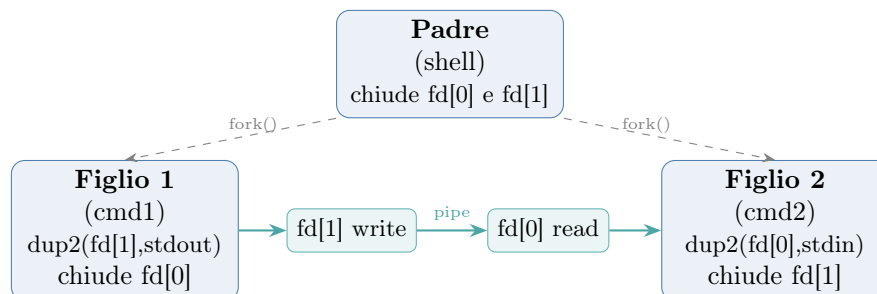
Esercizio 3: implementare `cmd1 | cmd2`

Estendi la shell per supportare l'operatore `|`. Se la riga contiene un singolo `|`, divide gli argomenti in due gruppi (`argv_left` e `argv_right`) e implementa la pipeline.

Il padre deve:

1. Chiamare `pipe(fd)` per creare il canale.
2. Fare `fork()` per il **primo figlio** (lato sinistro): reindirizza `stdout` su `fd[1]`, chiudi `fd[0]`, esegui il comando con `execvp()`.
3. Fare `fork()` per il **secondo figlio** (lato destro): reindirizza `stdin` su `fd[0]`, chiudi `fd[1]`, esegui il comando con `execvp()`.
4. Nel padre: chiudi **entrambi** i capi della pipe, poi attendi entrambi i figli con `waitpid()`.

Lo schema completo del flusso dei file descriptor:



[!] Il padre deve chiudere la pipe!

Dopo le due `fork()`, il padre possiede ancora copie di `fd[0]` e `fd[1]`. Se non le chiude, il secondo figlio non riceverà mai EOF quando il primo termina: il lato lettura della pipe rimarrà aperto perché il padre tiene ancora `fd[1]`. Il padre deve chiudere entrambi i capi *prima* di chiamare `waitpid()`.

► Come rilevare il `|`

Dopo il parsing, scorri `argv[]` cercando la stringa `"|"`. Se la trovi all'indice `i`, imposta `argv[i] = NULL` per troncare il primo vettore di argomenti; `argv_right = &argv[i+1]` è il secondo.

Test attesi:

```
minish> ls -l | grep .c
minish.c
minish> cat /etc/passwd | grep root
root:x:0:0:root:/root:/bin/bash
minish> echo uno due tre | wc -w
3
minish> ps aux | grep minish
user  9876  0.0  0.0  2048  512 pts/0  S+   10:01   0:00  ./minish
```

Compilazione e test

```
gcc -Wall -Wextra -o minish minish.c
./minish
```

Per verificare che non rimangano zombie:

```
# In un secondo terminale, mentre minish gira:
watch -n1 'ps aux | grep -E "minish|defunct"'

# Oppure, dentro minish stesso:
minish> sleep 2 &
[1] 1234
minish> ps aux | grep defunct
# Non deve apparire nulla dopo che sleep termina
```

Per tracciare le syscall e osservare fork/exec/wait in azione:

```
strace -f -e trace=fork,execve,wait4,pipe,dup2 ./minish
```

Il flag `-f` di `strace` segue anche i processi figli.